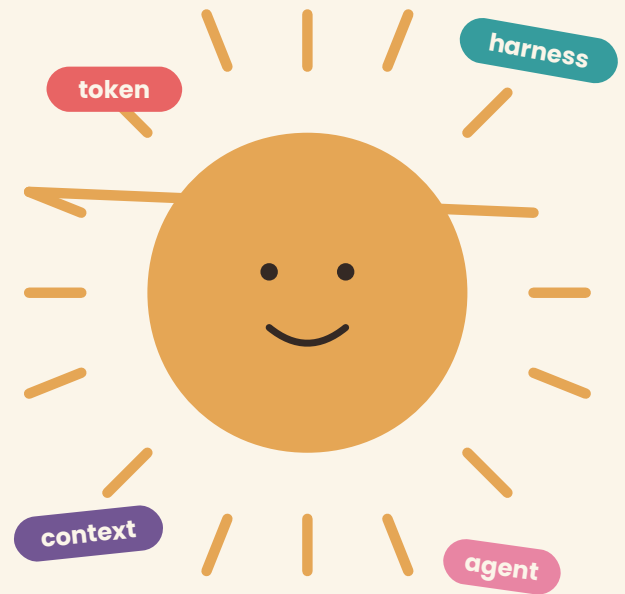




AN ILLUSTRATED FIELD GUIDE

The *AI Coding* *Dictionary*

Sixty terms of the trade, translated into plain English – with usage, opinions, and the occasional drawing.

Words by **Matt Pocock**

Source: github.com/mattpocock/dictionary-of-ai-coding

Illustrated PDF edition

WHY THIS EXISTS

A whole VC-funded economy benefits from keeping this hard to understand.

AI coding can feel like it's just for experts — unexplained jargon, mysterious failures, bills that don't seem to match the work.

It isn't, really. A lot of the confusion is *manufactured*. The basic terms of engagement are learnable in an afternoon. Once you have them, the whole thing stops feeling like guesswork.

Why does context degrade? Why is the bill so high? Why does the same prompt behave differently from one day to the next? Each has a clean answer, *once someone tells you the words to use*.

That's what this dictionary is for. **The vocabulary of AI coding, translated into plain English.**

WANT MORE THAN THE VOCABULARY?

Join 62,000+ developers at aihero.dev/newsletter for skills, thinking on AI engineering, and resources that'll keep you ahead of the curve.

How to read each entry

- **Term** — the headword, in its section's accent colour.
- **Definition** — what the term *actually* means, with cross-references in [tinted underline](#).
- **Usage** — a short overheard dialogue showing the term in the wild.
- **Avoid** — common mis-uses or vendor terms that lose precision.

CONTENTS

Seven sections, sixty terms.

01

The Model

What's actually predicting tokens.

14

02

Sessions, Context Windows & Turns

Where the conversation actually lives.

8

03

Tools & Environment

How an agent reaches into the world.

9

04

Failure Modes

Why it goes wrong, named precisely.

9

05

Handoffs

Passing context between sessions.

7

06

Memory & Steering

Persistence and progressive disclosure.

5

07

Patterns of Work

How humans and agents actually collaborate.

8

Two infographics inside: One request, three token streams & Anatomy of an Agent.

01

The Model

What's actually predicting tokens.

The frozen brain at the bottom of the stack. Billions of parameters, one job: predict the next token. Everything else — agents, tools, memory — is scaffolding around this.



01 The Model

Model

The parameters, Stateless — does next-token prediction and nothing else. “Claude Opus 4.7” and “GPT-5” are models. On its own a model can't do anything agentic; it has to be harnessed.

USAGE

- *Should we switch the model from Sonnet to Opus for the planning step?*
- Try it — but the harness is doing most of the lifting on this task. The model swap won't help if the system prompt and tools are wrong.

Parameters

The numbers inside a model — often billions of them — tuned during training. Everything the model “knows” lives in them. Training sets them; inference uses them unchanged. Also called *weights*.

USAGE

- *Can we fine-tune it on our codebase?*
- That'd update the parameters — different model afterwards. For one project it's almost always cheaper to load the codebase as context than to retrain.

Training

The process that sets a model's parameters, by exposing it to vast amounts of text and adjusting parameters to improve next-token prediction. A one-time, expensive process done by the model provider. Encompasses both pre-training (the bulk run) and post-training (later refinements like instruction-following and safety); the distinction doesn't matter at this glossary's level.

USAGE

- *Can we get it to know our internal API?*
- Not via training — that's a months-long process by the model provider. Load the API docs into context instead, that's the lever you actually have.

Inference

Running a trained model to generate output — what happens on every model provider request. Parameters stay fixed; the model just does next-token prediction over the context it's given. Cheap relative to training, but billed per token and the dominant cost of using a model.

USAGE

- Why does the bill scale with usage instead of being a flat license?
- You're paying for inference — every model provider request runs the model on the provider's hardware. Training already happened, but inference costs accrue per request, and a single turn can expand into many requests when tools are called.

Token

The atomic unit a model reads and writes. Roughly word-sized but not exactly — common words are one token, rare or long ones split into several. Context window size, cost, and latency are all counted in tokens.

Avoid: “word” — token boundaries don't match word boundaries, and tokens-per-second / tokens-per-dollar are the units that actually matter.

USAGE

- How big is this prompt going to be?
- Run it through the tokenizer — the schema's compact but the JSON keys are weird, so they'll split into more tokens than you think.

Next-token prediction

What the model actually does. Given a context, it samples one next token, appends it, and runs again. Every output — a sentence, a tool call, a thousand-line file — is built one token at a time. The model has no other mode of operation.

USAGE

- How does the agent ‘decide’ to call a tool?
- It doesn't — it's next-token prediction all the way down. The tool call is just a structured string the harness parses out of the output stream.

Non-determinism

The same input can produce different output. Run a model twice with identical context and you may get two different answers — sometimes a word, sometimes a completely different approach. Nothing in your code has to change for this to happen.

It's a property of how models generate text, and how model providers serve requests. There's no setting you can flip to make it go away.

Expect a spread of results from an agent on the same task. Some days the model will feel sharp; some days it'll feel like it's lost the plot. Same task, different rolls of the dice.

Be careful not to over-narrativize this. Humans are pattern-matching machines, and a string of bad runs can feel like proof that “the model got worse this week.” Usually it's just the distribution.

USAGE

- *Claude has been awful today. Did they ship a worse version?*
- Probably not — model output is non-deterministic. You're going to have good days and bad days on the same task. Try again tomorrow before you go looking for a cause.

Model provider

Whatever serves a model for inference. Usually a remote service (Anthropic, OpenAI, Google), but can also be local — Ollama, LM Studio, llama.cpp running on your own machine. The harness doesn't run the model itself; it asks a provider to.

USAGE

- *Can we run this offline for the air-gapped client?*
- Swap the model provider to a local one — Ollama or llama.cpp on their box. The harness doesn't care, it just hits a different endpoint.

Harness

Everything around the model that turns it into an agent: tools, system prompt, context-window management, permissions, hooks. **Claude.ai** and **Claude Code** run on the same model but behave differently because their harnesses differ.

USAGE

- Same model, why is Claude Code editing files and Claude.ai just answering questions?
- Different harnesses — Claude Code has filesystem tools, a different system prompt, and a permission layer. The model isn't the variable here.

Model provider request

One round-trip from the harness to the model provider. The harness sends the current context; the provider returns one response (a tool call or a final answer). A single user message can spawn many model provider requests if the agent calls tools — each tool result triggers another request.

USAGE

- One question burned forty thousand tokens?
- Look at the tool calls — twelve grep, eight read, four edits. Each tool result spawns another model provider request, and the whole session prefix re-sends every time.

Input tokens

Tokens the harness sends on each model provider request. Billed at a lower rate than output tokens.

USAGE

- Bill's high but the agent's barely writing anything.
- It's the input tokens — every turn re-sends the whole session. Without the prefix cache you re-pay for the history each request.

Output tokens

Tokens the model generates back. Billed at a higher rate than input tokens, since they cost more compute to produce.

USAGE

- *The refactor session is burning through credit even though the inputs are small.*
- Agent's rewriting whole files instead of patching. Output tokens cost roughly five times the input rate — get it emitting edits and the bill drops.

Prefix cache

The provider-side store that lets consecutive model provider requests skip re-processing a shared prefix. When the start of a request matches the start of a recent one — same system prompt, same history up to some point — the provider reuses its prior work and bills those tokens as cache tokens at a much lower rate.

Anything that changes the prefix (reordering files, rewriting the system prompt mid-session, injecting a timestamp near the top) invalidates the cache from that point on, and the rest of the request bills at full input token rate.

USAGE

- *Why did the bill spike halfway through the session?*
- Harness started injecting the current time into the system prompt every turn. Prefix cache breaks at the first changed token, so every request after that billed at full rate.

Cache tokens

Input tokens the provider has cached from a previous model provider request so it doesn't have to re-process them. When consecutive requests share a prefix, the provider reuses the work via its prefix cache and bills the cached portion at a much lower rate. The lever that makes long sessions affordable — without it, every turn re-pays for the whole history.

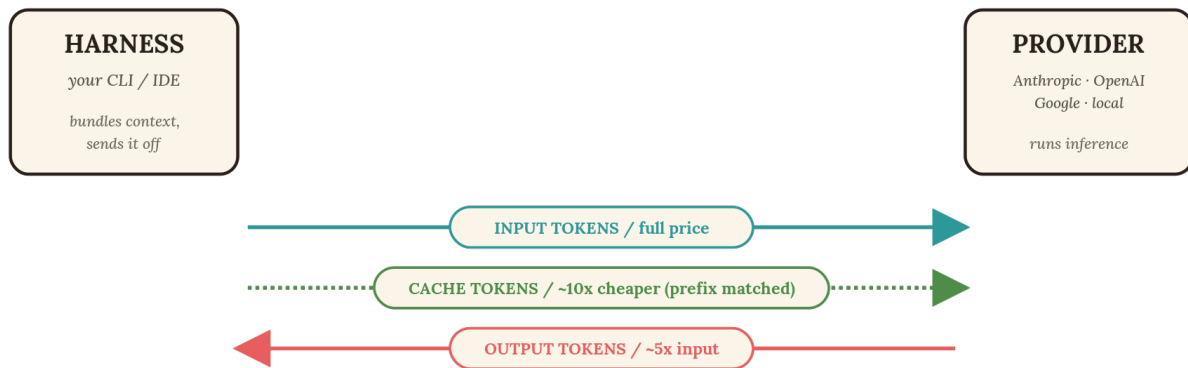
USAGE

- *Cost on long sessions is brutal — eight bucks for a refactor.*
- Check the cache tokens. If the harness is reordering the system prompt or files between turns, the prefix breaks and you re-pay full input rate every request.

One request, three token streams.

One request, three token streams

Every model provider request bills you in three buckets — and only one of them is cheap.



Relative cost per token

Rough industry shape — the absolute prices change, the ratios mostly don't.



Why prefix cache matters

Turn 4	system prompt	history so far	new msg	
Turn 5	cached	cached	new msg	+ ~10x cheaper
Turn 6	prompt + timestamp (!)	full input rate	full rate	(!) cache broke

One stray timestamp at the top invalidates everything after it. Keep the prefix stable.

How the concepts in Section 01 fit together.

02

Sessions, Context Windows & Turns

Where the conversation actually lives.

The model has no memory. A session does — it accumulates messages, tool results, and read files until it fills the context window and starts forgetting. Knowing the units is half the battle.



§2

02 Sessions, Context Windows & Turns

Stateless

Carries no information forward. The model is stateless across model provider requests — each request resends the full context window, because the model has no way to see anything else. An agent is stateless across sessions by default: a new session starts empty, with no trace of prior ones. Counterpart to stateful.

USAGE

- *Why does it forget the convention every time I clear?*
- The model's stateless — the new session starts empty. If you want it carried, write it to AGENTS.md or a memory file the harness loads at session start.

Context

The relevant information the agent has access to right now. The abstract noun — not the raw input the model sees (that's the context window), not the running history (that's the session), but *what the agent knows that's pertinent to the task*. “Loading something into context” means making it part of this set; “context engineering” is the discipline of curating it.

USAGE

- *It keeps inventing fields that aren't in the type.*
- The type file isn't in context — it's reading the call sites and guessing. Read the definition in first.

Context window

Everything the model sees on each model provider request. Finite, model-specific, and the only surface through which the model perceives anything.

Avoid: “memory” — the context window is working state and doesn't persist across sessions. Memory is a separate concept layered on top.

USAGE

- Can I just paste the whole monorepo into the prompt?
- The context window's 200k tokens — that's maybe a fifth of the repo. Pick the files the task touches, leave the rest behind a tool call.

Stateful

Carries information forward. A session is stateful across turns — context accumulates as the session runs, which is why long sessions drift into the dumb zone. An agent can be made stateful across **sessions** by adding a memory system that persists information into the environment and reloads it at the start of future sessions. The model is never stateful; any apparent continuity is the harness re-feeding context. Counterpart to stateless.

USAGE

- *It remembered my preferences from yesterday — does that mean the model learned them?*
- No, the agent's stateful because the harness wrote them to a memory file and reloaded them at session start. The model itself saw nothing of yesterday.

Agent

A model harnessed with tools, a system prompt, and a context window, that takes turns with a user. *Claude Code is an agent. Cursor is an agent. Claude.ai is an agent.* An agent is what you actually talk to — it's the model in motion, configured for a purpose.

Avoid: “the AI”, “the bot” (too vague — they hide whether you mean the parameters or the harnessed thing).

USAGE

- Which agent are you using for the migration?
- Claude Code locally, Cursor for the UI work — same model underneath, different harnesses.

System prompt

The instructions the harness prepends to every model provider request — the agent's standing brief: who it is, how to behave, which tools it can call, what conventions to follow. Usually stable across a session.

USAGE

- Two harnesses, same model, totally different behavior on the same prompt.
- Different system prompts. One's tuned for terse code edits, the other for explaining — that's where the divergence lives, before your message even arrives.

Session

One bounded run of interaction with an agent. Starts empty, accumulates messages, tool results, and files read, and ends when cleared, closed, or compacted into a fresh session. The session is what *fills* the context window: if the context window is the box, the session is the stuff slowly filling it up. Work too large for a single context window must be split across sessions.

USAGE

- How long can one session run before it falls apart?
- Depends on the work — a focused refactor stays sharp longer than open-ended research. Once the session bloats, hand off or compact, don't push through.

Turn

One user message plus everything the agent does in response, up until it yields back to the user. Contains one or more model provider requests — many, if the agent calls tools. A clarifying question closes the turn; your reply opens the next one. The hierarchy is **Session > Turn > Model provider request**.

USAGE

- One turn took two minutes?
- It made fourteen tool calls inside that turn — each one is a separate model provider request. Latency stacks up before the agent finally yields back to you.

03

Tools & Environment

How an agent reaches into the world.

An agent perceives nothing on its own. Every file it reads, every command it runs, every API it touches goes through a tool call to the environment — and a tool result back.



§3

03 Tools & Environment

Environment

The world the agent acts on — anything outside the harness that the agent perceives through tool results and changes through tool calls. The harness *runs* the agent; the environment is what the agent *works in*. A file like `AGENTS.md` lives in the environment; the harness is what loads it into the context window. A filesystem is the most common kind of environment, but not the only one (a database, a remote API, a browser session can all be environments).

Avoid: using “environment” for the runtime or the harness itself — the harness is the wrapper, the environment is the workspace.

USAGE

- *The agent can't see the staging DB schema.*
- Wire it into the environment — give it a `psql` tool scoped to read-only on staging. The harness is fine, it just has nothing to act on.

Filesystem

A tree of files and directories the agent reads from, writes to, and executes within — the default kind of environment for a coding agent. AGENTS.md, skills, source code, build scripts, and tool configs all live in a filesystem. When a harness “starts in your project,” it's pointing the agent at a filesystem.

USAGE

- *Why isn't it picking up my AGENTS.md?*
- It's running against a different filesystem — the sandbox mounted the parent dir, not the project root. Repoint the harness.

Tool

A function the harness exposes for the agent to call — Read, Write, Bash, Search. Tools are how an agent perceives and acts on the environment: it can't see the environment except through tool results, and can't change it except through tool calls. Each tool call costs an extra model provider request, since the result has to go back to the model before it can decide what to do next.

USAGE

- Can the agent query staging directly?
- Add a `psql` tool to the harness, scoped read-only on staging. Without a tool for it, the agent's blind to anything outside the filesystem.

Tool call

The model's output naming a tool and its arguments — just structured text. It doesn't do anything on its own; the harness has to read it and execute. Produced by the model in one model provider request.

USAGE

- *It said it ran the tests but the file timestamps haven't changed.*
- Look at the transcript — did it actually emit a tool call, or just describe running them? The model produces the call, but if the harness didn't execute it, nothing happened.

Tool result

What the harness sends back after executing a tool call — the file contents, the command output, the error. The agent's only window onto the environment. Travels back to the model in the next model provider request, where the model decides what to do with it. Tool call and tool result are two ends of the same exchange, both inside one turn.

USAGE

- *It's reasoning about the file like it's empty.*
- The tool result came back as a permission denial, not the contents. The model only saw the error string — it has no other window onto the file.

Permission request

What the harness shows the user before executing a tool call that isn't pre-approved. The model produces a tool call; instead of running it immediately, the harness pauses and asks. Approve and it runs; deny and the harness reports the denial back to the model as a tool result. The mechanism by which a harness puts a human in the loop for risky or sensitive actions.

USAGE

- *It's been blocked on a permission request for ten minutes – I was in a meeting.*
- That's the cost of human-in-the-loop. Pre-approve the safe tools so the request only fires on the actually-risky calls.

Permission mode

The permission-gating slice of an agent mode – which tool calls trigger a permission request and which run automatically. The original purpose of mode systems before harnesses started bundling behavioral instructions on top.

USAGE

- *It paused on every grep – totally killed the AFK run.*
- Loosen the permission mode for read-only tools, keep prompting on writes and shell. Most permission requests on a research session are noise.

Agent mode

A preset that shapes how the agent operates at runtime — bundles a permission mode with behavioral instructions injected into the system prompt. Examples: a default that prompts on risky calls, a **plan mode** that blocks edits and steers the agent toward research, an **accept-edits** mode that auto-approves edits, a **bypass permissions** mode (colloquially **YOLO mode**) that auto-approves everything. Can flip mid-session.

Vendor terms: Claude Code calls these “permission modes,” Codex calls them “approval modes” — both predate behavioral bundling.

USAGE

- *It keeps editing files when I just want a plan.*
- Switch to plan mode — it'll block writes and stay in research.
- *What about for the AFK run later?*
- Bypass mode, but only inside the sandbox.

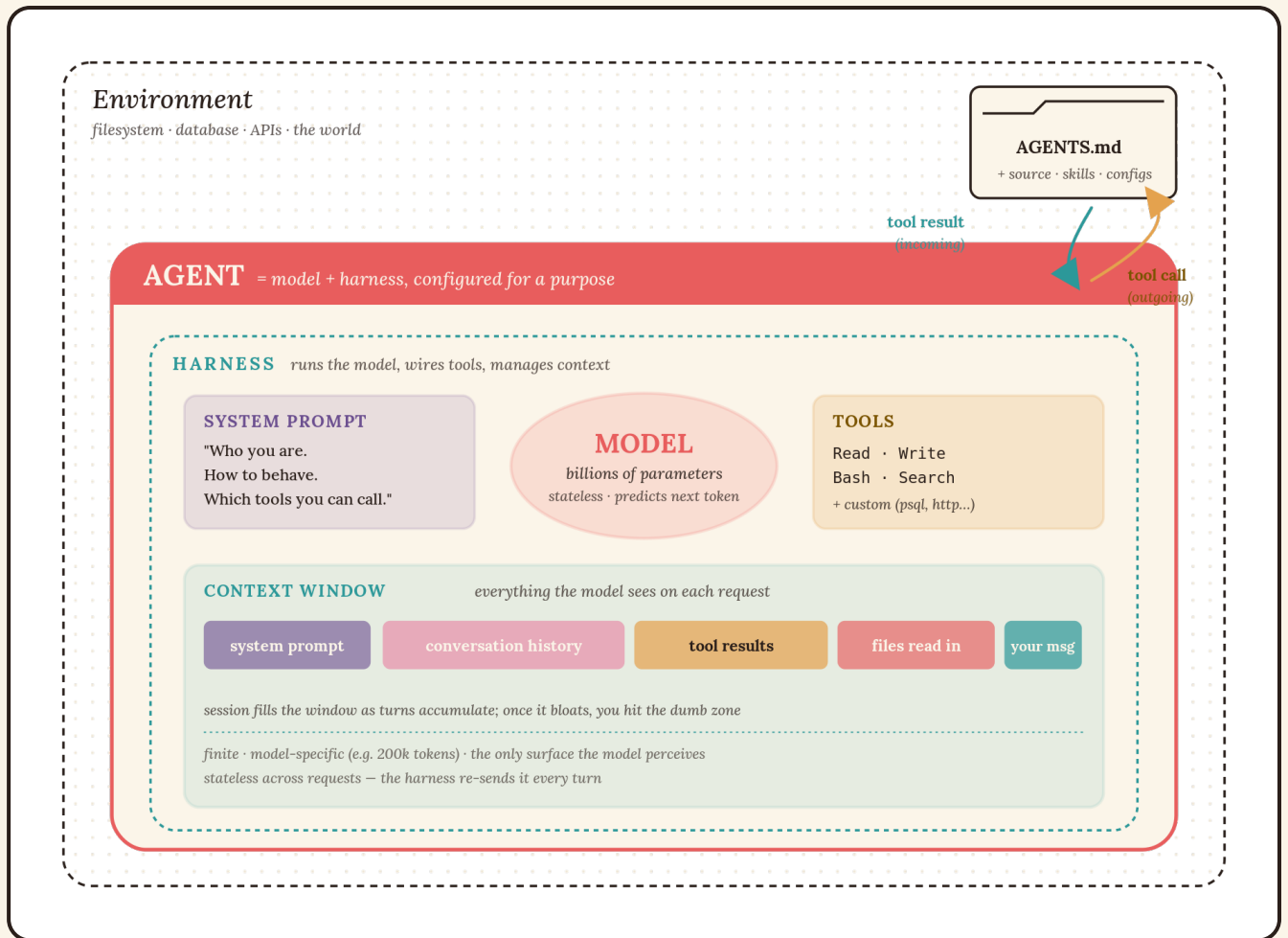
Sandbox

An isolated environment the agent runs inside — a container, VM, ephemeral filesystem, or restricted-permission shell. Limits the blast radius of agent actions: even if the agent runs destructive commands or fetches something malicious, the damage is contained. The safety substrate that makes AFK practical.

USAGE

- *I want to let it run bypass-permissions overnight but I'm not ready for that.*
- Put it in a sandbox — fresh container, no credentials mounted, no network out. Worst case it nukes its own filesystem and you discard the container.

Anatomy of an Agent.



How the concepts in Section 03 fit together.

04

Failure Modes

Why it goes wrong, named precisely.

“The AI got worse” is usually a vague way of saying something specific. Sycophancy. Faithfulness hallucination. Attention degradation. Naming the failure is the first step to fixing it.



§4

04 Failure Modes

Sycophancy

Confidently agreeable model output. Caused by training: the model was shaped to favor answers humans liked, and humans tend to like agreement more than they like being told they're wrong. So the model learned that agreeing is rewarded — even when the agreement is incorrect.

Surfaces as:

- *Caving under pushback* — reverses a correct answer when you say “are you sure?”.
- *Praising bad input* — agrees your broken plan is brilliant before analysing it.
- *Biased framing* — review skews positive when you signal you wrote it; negative when you signal someone else did.
- *Mimicry* — repeats your mistakes back to you as confirmation.

Diagnostic test: would the model have said this without your steer? If the only thing that changed was your tone or framing, it's sycophancy, not a real shift in analysis.

Fix: hide your preferences. Phrase prompts neutrally — “review this code” not “is this code good?”.

Avoid: using “sycophancy” for any wrong answer that happens to please you.

USAGE

- *It said my refactor plan looked great, then I asked ‘are you sure?’ and it walked the whole thing back.*
- Classic sycophancy — it agreed first because you sounded confident, then caved because you sounded doubtful. The plan's quality didn't change, your tone did. Clear and re-ask without signalling either way.

Hallucination

Confidently-wrong model output. Two flavors with different causes and fixes:

- *Factuality hallucination* — invented or wrong facts about the world (a function that doesn't exist, a wrong API signature, a fake citation). Caused by parametric knowledge gaps, often past the knowledge cutoff. **Fix:** load the right contextual knowledge.
- *Faithfulness hallucination* — output drifts from the contextual knowledge that's loaded, the user's instructions, or the model's own prior reasoning. Symptom of attention degradation; worsens in the dumb zone. **Fix:** clear or compact.

Avoid: “hallucination” as a bare synonym for “wrong” — without naming the flavor, the term has no diagnostic value.

USAGE

- *It hallucinated a `parseAsync` method on the schema.*
- Factuality or faithfulness?
- *The method exists in the docs I pasted — it just stopped reading them after turn forty.*
- Faithfulness then. Compact and reload, don't bother adding more docs.

Parametric knowledge

What the model “knows” from training, stored in its parameters. Frozen at training time — the model can't see its own parameters or update them. Detail is lost in the squeeze: billions of facts cram into a fixed number of parameters, and the rare ones blur. Source of fluency on common topics, and of fabrication on uncommon ones. Counterpart to contextual knowledge.

USAGE

- *It writes flawless React but invents methods on our internal SDK.*
- React is dense in the parametric knowledge — millions of training examples. Your SDK isn't, so the model fills in plausible-looking shapes. Load the SDK docs into context.

Knowledge cutoff

The date past which a model has no parametric knowledge. Libraries, APIs, and events from after the cutoff are fabrication traps unless their docs are loaded as contextual knowledge. Each model release ships with its own cutoff.

USAGE

- It keeps writing the v3 SDK syntax – we're on v5.
- v5 shipped after the knowledge cutoff. Load the v5 changelog as contextual knowledge, otherwise it'll keep fabricating from the older parametric version.

Contextual knowledge

Facts the agent can read directly from the context right now – the user's task, files the agent has read in, tool results, AGENTS.md content loaded at session start. Counterpart to parametric knowledge: parametric is *recalled* from the parameters; contextual is *read* from the window. Hallucinations are much less common when the agent works from contextual knowledge – the answer is right in front of it, not dredged up from a blurred memory.

Reach for this term only when contrasting with parametric knowledge; otherwise just say **context**.

USAGE

- Why does it nail the API when I paste the docs and fabricate it when I don't?
- With the docs in, it's contextual knowledge – reading off the page. Without, it's parametric and the rare endpoints blur.

Attention relationship

When predicting each token, the model factors in every other token in the context – some heavily, others barely at all. The pairing between two tokens is an **attention relationship**, and meaningful pairs (“her” with “Sarah”, or a `getUser()` call with its `function getUser` definition) influence each other more than unrelated ones. A context of N tokens has on the order of N^2 relationships.

USAGE

- It keeps confusing the two *user* symbols across the diff – sounds like we're in the dumb zone.
- Yeah, the attention relationship between each call site and its declaration is fighting the other one – same token shape, different bindings. Rename one and the pairings sharpen.

Attention budget

Each token has a finite amount of influence to distribute across the rest of the context. Heavy influence on one relationship leaves less for others. The budget is per-token and doesn't grow when the context does, which is why long sessions dilute.

USAGE

- *Why does it keep ignoring the schema I pasted at the top?*
- We're well into the dumb zone — every token's attention budget is fixed, but the context kept growing. The signal on the schema is now competing with thousands of newer tokens.

Attention degradation

As a session grows, each token's attention budget is spread across more competitors. The signal on any one meaningful relationship shrinks; noise from irrelevant context crowds in. Same model, same parameters — just more mouths to feed from the same plate. Cause of the smart zone / dumb zone effect.

USAGE

- *It's deep in the dumb zone — inventing generics that aren't in the type file.*
- Attention degradation. The type definitions are still in context, but the signal on them is buried under everything we've added since. Clear and reload.

Smart zone

Early in a session the agent is in a “smart zone” — sharp, focused, recall is good. As the session grows it drifts into a “dumb zone”: sloppier, forgetful, more mistakes — and more faithfulness hallucinations. Same model, same harness — just more context. The felt effect of attention degradation. On frontier models, the dumb zone commonly begins around 100,000 tokens — though this is debated. Clear or compact when the session bloats; don't push through.

USAGE

- *It nailed the first three components and just butchered the fourth.*
- You're out of the smart zone — same model, just deep into the dumb zone now. Compact and reload the plan, the next component will land.

05

Handoffs

Passing context between sessions.

Work too large for one context window has to be split.

Clearing, compacting, spec → ticket → session — the vocabulary for moving state without losing the plot.



05 Handoffs

Clearing

Ending the current session and starting a fresh one. The next message begins with an empty session and an empty context window. Usually user-driven.

USAGE

- It's stuck looping on the failing test.
- Just clear it — start a fresh session with the plan doc and the test file. No point fighting the existing context.

Handoff

Transferring agent context from one session to another, with no return path. The carry mechanism varies — a written handoff artifact, an in-memory summary (compaction), and others. Distinct from clearing (no transfer at all). Reasons vary: switching roles (planner → implementer), kicking off an AFK run, fanning out to parallel sessions, or freeing up context window room.

USAGE

- Planning session is getting heavy — should I just keep going?
- Do a handoff. Write the decisions to a doc, clear, start the implementation in a fresh session reading from it.

Handoff artifact

A document used as the carry mechanism for a handoff — written by one session to be read by another. One way among several (see also compaction).

USAGE

- How do I split this between the planning agent and the implementing one?
- Have the planner write a handoff artifact — file paths, decisions, constraints. The implementer's session opens with a pointer to the artifact and works from it as its brief.

Spec

A handoff artifact describing a multi-session piece of work — what's being built, not how each session does its share. Mutates as work progresses. Made of tickets.

USAGE

- *Should this all be one session?*
- No, write it up as a spec — break it into tickets, run each one in its own session. Trying to do the whole thing in a single context will hit the dumb zone before you're halfway.

Ticket

A handoff artifact scoping one session of work. Stands alone, or hangs off a spec as one of its children. Tickets can block or be blocked by sibling tickets, so the order of work falls out of their dependency graph rather than a linear plan.

USAGE

- *Where do I start on the migration spec?*
- Look at the ticket graph — the schema change blocks the backfill, the backfill blocks the API switch. Pick a leaf and run a session on it.

Compaction

A handoff done in-memory: the previous session's history is summarised and seeds a fresh session. Lossy — detail traded for headroom. Triggered manually by the user, or automatically.

USAGE

- *Context's getting heavy and I still have the test pass to do.*
- Compact before you start — write what's load-bearing into the summary prompt so the new session keeps the schema decisions and drops the exploration.

Autocompact

Compaction triggered automatically by the harness when the context window approaches full.

USAGE

- It *doesn't seem to remember what we decided about the schema earlier*.
- Autocompact fired between turns — the early decisions got summarised and we must have lost something. Reload the plan doc, or compact manually next time so you control what gets kept.

06

Memory & Steering

Persistence and progressive disclosure.

The model is stateless. The agent can be made to feel otherwise — with files in the environment loaded at session start, and skills that show up only when needed.



§6

06 Memory & Steering

Memory system

A system that attempts to make an agent stateful across sessions. Persists information into the environment during a session and reloads it into the context window at the start of future ones, so the agent carries continuity beyond the user clearing the session.

USAGE

- I keep having to re-tell it I'm on Postgres, not MySQL.
- Wire up a memory system — write what it learns to the filesystem on the first turn, reload it at session start. The model itself is stateless; the memory layer fakes continuity.

AGENTS.md

A file in the environment that the harness loads into the context window at session start — the project's standing brief to the agent. Cross-harness convention.

Avoid: using AGENTS.md for content that should be progressively disclosed — anything in it pays a token cost every turn.

USAGE

- Why is every session starting with 4k tokens already burned?
- Check AGENTS.md — someone pasted the entire style guide in there instead of putting it behind a skill.

Progressive disclosure

Loading only the context an agent needs right now, with pointers to the rest. Borrowed from UI design.

USAGE

- Should I dump the entire style guide into AGENTS.md?
- No — progressive disclosure. Reference the style guide as a skill the agent loads when it actually needs to write a component. AGENTS.md pays the token cost every turn.

Skill

A teachable capability bundled as a unit — instructions and resources for doing one task well, kept in the environment and loaded into the context window only when relevant. The unit of progressive disclosure in a harness.

Avoid: “tool” — a tool is what the agent *calls*; a skill is instructions it *reads*.

USAGE

- *Where should I put the deploy runbook?*
- As a skill — the agent loads it only when the task involves deploys. In AGENTS.md it'd burn tokens on every turn for something we use weekly.

Subagent

An agent spawned by another agent via a tool call. Runs in its own session with its own context window, and reports a single tool result back. Distinct from a handoff — the parent specifically expects a return; a handoff has no return path. **Cannot spawn further subagents** — the tree is one level deep. Subagents exist to isolate context, not to compose hierarchies.

USAGE

- *The grep results are blowing out my context.*
- Spawn a subagent to do the search — it'll burn its own context window on the noise and report back the two file paths you actually need.

07

Patterns of Work

How humans and agents actually collaborate.

AFK runs. Vibe coding. Grilling. The named shapes of human→agent collaboration — each with its own assumptions about review, risk, and throughput.



§7

07 Patterns of Work

Human-in-the-loop

A working pattern where one or more humans pair with the agent during a session — reviewing, redirecting, or collaborating in real time. The human is present and engaged, not just gating individual actions.

USAGE

- *Run this AFK overnight?*
- No, schema migration — keep it human-in-the-loop. I want to see each step and steer if it picks the wrong column to backfill from.

AFK

A working pattern where the user kicks off a session and leaves the agent to run unattended. The throughput multiplier of AI coding — many AFK sessions can run in parallel while you sleep, eat, or work on something else. Usually requires a permissive permission mode plus sandboxing to be safe.

Avoid: “background agent” — centers the machine (“running in the background”) rather than the human pattern (“user has walked away”). AFK is the load-bearing fact: the user isn't watching.

USAGE

- *I'm running this AFK — three sandboxed agents on the refactor, reviewing the PRs in the morning.*
- Bypass permissions?
- *Yeah, read-only filesystem, no network.*

Automated check

A deterministic verification that runs in the environment — tests, type checks, lints, build, pre-commit hooks. Pass/fail, no judgement. The signal an agent can self-correct from without involving anyone else. A flaky test is a broken check, not a non-check; automated checks are deterministic *by design*.

Avoid: “feedback loop” / “backpressure” — both lump checks together with review. “test” — tests are automated checks, but not all automated checks are tests.

USAGE

- *The agent keeps shipping broken code in the AFK runs.*
- What automated checks are wired into the sandbox?
- *Just the unit tests.*
- Add typecheck and lint — it'll self-correct from those before the PR ever lands.

Automated review

An agent reviewing another agent's work, often with a different model or system prompt. Non-deterministic: it forms a judgement. Runs anywhere — pre-merge on a PR, post-hoc on commit history, mid-session as a subagent. An LLM-as-judge in CI is automated review, not an automated check; what the assertion *does* decides the category, not where it runs.

Avoid: “AI review” / “agent review” — too vague to distinguish from the working agent itself.

USAGE

- *We're getting too many bad PRs from the AFK runs.*
- Add an automated review step before merge — different model, separate system prompt, scoped to security and contract changes.

Human review

The user reading the code the agent produced and forming a judgement on it. Reading the diff or the changed files counts; reading the agent's *description* of what it did does not — narration is not the artifact.

Avoid: “code review” alone — ambiguous between human and automated.

USAGE

- *I human-reviewed the AFK output.*
- *You read the diff or just the summary?*
- *Diff. The summary said it deleted dead code — turned out the function was called from a generated file.*

Vibe coding

A working pattern where the user accepts the agent's code without human review. The diff is treated as opaque — what matters is whether the program behaves, not what's inside. Automated review and automated checks may still run; vibe coding is silent on both.

Avoid: “vibe coding” as a synonym for “low-quality AI coding” — the term names the review stance, not the resulting code.

USAGE

- *Did you read what it changed in the auth flow?*
- *Vibe coded it — login still works, that's all I checked.*
- *Read the diff before you push, vibing on auth is how secrets leak into logs.*

Design concept

The shared understanding of what's being built, held in common between user and agent but separate from any asset. Brookes' term (*The Design of Design*): the conversation, handoff artifacts, and the code are all assets that try to capture or reach the design concept, but none of them *are* it. Quality of the design concept is felt through the quality of the conversation that built it.

USAGE

- *It's writing exactly what I asked for and it's still wrong.*
- You don't share a design concept yet — it's filling gaps with assumptions. Keep talking until cancellation, refunds, and partial fulfilment all line up between you before you let it write a spec.

Grilling

A technique for developing a design concept with an agent: the agent interviews the user Socratically, one decision at a time, proposing a recommended answer for each. Slows the rush to a finished plan — no handoff artifact is written until the concept stabilises.

USAGE

- *It went straight to writing the spec and got the cancellation logic wrong.*
- Grill it first — make it ask you about partial cancels, refunds, and timing before it commits anything to the doc. Cheaper to resolve in conversation than in code.

That's the vocabulary.

Sixty words, seven sections. Enough to read most posts,
follow most debates, and call out most marketing.

For more, keep going at [github.com/mattpocock/
dictionary-of-ai-coding](https://github.com/mattpocock/dictionary-of-ai-coding) — the source repo updates as the
field does.

